Microservices Common Interview Questions (Beginner → Expert)

Oral / Conceptual Questions

Microservices Fundamentals

- What is microservices architecture?
- Why microservices over monolith? What are the tradeoffs?
- What is a modular monolith and when is it better than microservices?
- What is a distributed monolith? How do you identify it?
- How do you decide service boundaries?
- How do you apply DDD bounded contexts in microservices?
- How does Conway's Law influence microservices design?
- What are common microservices anti-patterns?

API Design & Contracts

- How do you design REST APIs for microservices?
- REST vs gRPC: when and why?
- How do you version APIs?
- How do you maintain backward compatibility?
- What is consumer-driven contract testing?
- How do you handle breaking changes in APIs?
- How do you design error responses consistently across services?
- What is HATEOAS and do you use it?
- How do you design pagination, filtering, sorting for APIs?
- How do you design idempotent APIs?

Communication Patterns

- Synchronous vs asynchronous communication: when to choose each?
- Request/response vs event-driven: when to choose each?
- What is event-driven architecture?
- What is pub/sub vs message queue?
- What is the outbox pattern and why is it needed?
- How do you manage schema evolution for events?
- How do you prevent chatty service communication?

Data & Transactions

- Why database-per-service?
- When is shared database acceptable (if ever)?
- How do you manage distributed transactions without 2PC?
- What is the Saga pattern?
- Saga choreography vs orchestration: compare.
- How do you handle compensation actions?
- What is CQRS and when do you use it?
- What is event sourcing and when do you avoid it?
- How do you handle reporting/analytics across microservices?
- How do you handle data duplication and source of truth?

Consistency, Ordering, Idempotency

- What is eventual consistency? Where is it acceptable/unacceptable?
- At-most-once vs at-least-once vs exactly-once delivery: explain.
- What is idempotency and how do you implement it?
- How do you handle duplicate messages/events?
- How do you handle out-of-order events?
- How do you ensure ordering in Kafka (partitioning strategy)?
- What is a DLQ and when do you use it?
- How do you handle poison messages?
- How do you design retry strategies without causing a retry storm?

Resilience & Fault Tolerance

- What is a circuit breaker? How does it work?
- What is bulkhead isolation?
- What is rate limiting and where do you apply it?
- What timeout strategy do you use for service-to-service calls?
- How do you choose retry policies?
- How do you design graceful degradation?
- What is backpressure and how do you implement it?

Service Discovery & Routing

- What is service discovery? Client-side vs server-side discovery?
- What is an API Gateway and what belongs there?
- Gateway vs BFF: when and why?
- How do you do API composition/aggregation?
- How do you handle cross-cutting concerns (auth, logging, throttling)?

Security

- OAuth2 vs OIDC: what's the difference?
- JWT vs opaque tokens: tradeoffs and revocation strategy?
- How do you do service-to-service authentication?
- What is mTLS and why is it used?
- How do you manage secrets (rotation, storage, access)?
- How do you implement authorization (RBAC/ABAC) in microservices?
- How do you secure internal endpoints?
- How do you prevent data leakage across tenants (multi-tenancy)?

Observability

- What is observability vs monitoring?
- What are logs/metrics/traces and how do they work together?
- What is a correlation ID and how do you propagate it?
- What is distributed tracing and how do you use OpenTelemetry?
- What are SLIs/SLOs/SLAs?
- What is RED method? What is USE method?
- How do you design alerting to avoid noise?
- How do you debug a production issue spanning multiple services?

Deployment & Release Engineering

- Blue/green vs canary vs rolling deployment: compare.
- What is feature flagging and how do you use it safely?
- How do you roll back safely in microservices?
- How do you do zero-downtime DB migrations?
- How do you handle config management across environments?
- What is GitOps?
- How do you handle secrets in CI/CD pipelines?

Kubernetes / Container Platform

- What is a pod, deployment, service, ingress?
- Readiness vs liveness vs startup probes: differences?
- Requests vs limits: impact on performance and stability?
- What is HPA and when does it fail?
- How do you debug CrashLoopBackOff?
- How do you handle node/pod failures?
- What are network policies and why do they matter?

Service Mesh

- What problems does a service mesh solve?
- When is a service mesh not worth it?
- What are sidecars and what overhead do they introduce?
- Should retries/timeouts be handled at mesh or application layer?

Performance & Scalability

- How do you reduce p95/p99 latency across microservices?
- What is tail latency and why does it matter?
- What caching layers do you use (client/CDN/app/DB)?
- Cache-aside vs write-through vs write-behind: compare.
- How do you handle cache invalidation?
- How do you do load testing and capacity planning?

Testing Strategy

- Unit vs integration vs contract vs E2E tests: what goes where?
- How do you test asynchronous flows?
- How do you test Sagas?
- What is test data management strategy in microservices?
- How do you use test containers?
- How do you avoid flaky tests?

Multi-Region / DR

- Multi-AZ vs multi-region: difference?
- Active-active vs active-passive: when to use which?
- What are RPO and RTO?
- How do you test failover (game days/chaos engineering)?

System Design Prompts (Oral)

- Design an order management system with inventory, payment, shipping.
- Design a payment system focusing on idempotency and reconciliation.
- Design an event-driven notification system with retries and DLQ.
- Design a rate-limited public API platform with API keys and quotas.
- Design a multi-tenant SaaS backend with isolation and observability.
- Design a real-time analytics pipeline using Kafka and stream processing.

Coding / Practical Implementation Questions

REST & Spring Boot Microservices

- Implement a REST API with validation, pagination, and error handling.
- Implement global exception handling with consistent error schema.
- Implement an idempotent POST endpoint using an idempotency key.
- Implement request correlation ID propagation across services.
- Implement OpenAPI/Swagger and enforce schema validation.

Resilience (Retries/Circuit Breaker/Timeout)

- Implement timeouts and retries for outbound HTTP calls.
- Implement circuit breaker + fallback for a downstream service.
- Implement bulkhead isolation for expensive external calls.
- Implement rate limiting per user/API key.

Messaging (Kafka/RabbitMQ)

- Implement a Kafka producer with schema/version strategy.
- Implement a Kafka consumer with idempotent processing.
- Implement consumer retry + DLQ handling.
- Implement message deduplication using a store (Redis/DB) with TTL.
- Implement ordering guarantees using partition keys.
- Implement exactly-once-like processing using transactional outbox + consumer idempotency.

Outbox / Inbox Patterns

- Implement transactional outbox table write within the same DB transaction.
- Implement outbox publisher worker that publishes and marks processed.
- Implement inbox pattern for consumers to avoid duplicate processing.

Saga (Orchestration/Choreography)

- Implement Saga orchestration with state machine persisted in DB.
- Implement compensation logic for a failed multi-step workflow.
- Implement Saga choreography with events and compensating events.

Data Consistency & CQRS

- Implement CQRS with separate write model and read projection.
- Build a materialized view from events and handle rebuild/replay.
- Implement optimistic locking to prevent lost updates.

Security

- Implement OAuth2 resource server with JWT validation.
- Implement role-based access control on endpoints.
- Implement service-to-service auth (mTLS or signed tokens).
- Implement secure secret loading without hardcoding.

Observability

- Emit structured logs with correlation IDs.
- Add custom metrics (latency, error rate, saturation).
- Add distributed tracing instrumentation for HTTP and Kafka flows.
- Implement audit logging with immutability requirements.

Kubernetes Readiness & Graceful Shutdown

- Implement readiness/liveness endpoints correctly.
- Implement graceful shutdown handling in Spring Boot (stop accepting traffic, finish in-flight).
- Implement health checks that reflect downstream dependencies safely.

Performance

- Implement caching with cache-aside and proper invalidation.
- Implement a high-throughput endpoint and address thread/connection pool tuning.
- Implement async processing to avoid blocking request threads.

Testing

- Write contract tests for producer/consumer APIs.
- Write integration tests using test containers for DB/Kafka.
- Write tests for idempotency and duplicate event handling.
- Write tests for saga compensation flows.

Outdated (Label as Outdated if asked)

Outdated (still sometimes asked)

- Outdated: Explain SOA vs Microservices (as the main focus).
- Outdated: ESB-centric integration patterns for microservices.
- Outdated: Netflix OSS stack as default (Zuul/Hystrix/Eureka) in all cases.